

Searched for regex `class (OperationsDefinition|ProcessSegment|SegmentRequirement|OperationsRequest|EquipmentRe`
(`**/server/src/mes/**/*`.py), 3 results

Searched for regex `^class |route_id|product_id|ForeignKey (`
`**/server/src/mes/core/production/models.py`), 8 results

Read , lines 280 to 459

Quick note before the diagram: this codebase doesn't have separate tables named `OperationsDefinition` / `ProcessSegment` / `SegmentRequirement` / `OperationsRequest`. Those ISA-95 Part 2 concepts map onto the existing models like this:

ISA-95 Part 2	Implemented as
Operations Definition	<code>ProcessRoute</code>
Process Segment	<code>RouteStep</code>
Segment Requirement	<i>(implicit — requirements hang directly off <code>RouteStep</code>)</i>
Equipment Requirement	<code>StepEquipmentRequirement (+ RouteStep.equipment_class_id)</code>
Material Requirement	<code>StepMaterialRequirement</code>
Operations Request	<code>ProductionOrder</code>
Equipment Class	<code>EquipmentClass (+ EquipmentClassProperty)</code>
Equipment Capability	<code>EquipmentCapability (+ EquipmentCapabilityProperty)</code>

ER diagram using those mappings:

Entities are labeled with both their ISA-95 Part 2 name and the actual class used in the code (e.g. `ProcessRoute_OpsDefinition = ProcessRoute` playing the role of *Operations Definition*).

Key relationships:

- **Operations Definition → Process Segments:** `ProcessRoute` owns an ordered list of `RouteSteps`; allowed materials live on `RouteMaterialAssignment`.
- **Segment Requirements** (attached directly to the `RouteStep`):
 - *Equipment requirement (class)* — `RouteStep.equipment_class_id` → `EquipmentClass` (ISA-95 preferred form).
 - *Equipment requirement (specific)* — `StepEquipmentRequirement` → a specific `Equipment` (with `use_type` = required/preferred/alternate).
 - *Material requirement* — `StepMaterialRequirement` → `MaterialDefinition`.
- **Equipment Class model:** `EquipmentClass` ↔ `EquipmentClassProperty` (typed spec), classifies `Equipment`, and anchors `EquipmentCapability` instances. `EquipmentCapabilityProperty` stores the actual value for each class-level property.
- **Operations Request:** `ProductionOrder` references both a `ProductDefinition` and an optional `ProcessRoute` (falls back to the product's default route at runtime — see `routing/service.py`).

If you want, I can save this as `.mmd` + regenerate a PDF alongside the existing `routing_physical_er.pdf` — just say the word.